Chandubhai S Patel Institute of Technology

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical Statement: To automate testing and deployment of the application using a CI/CD pipeline.

Screenshots (Supplementary Problems are also implemented):

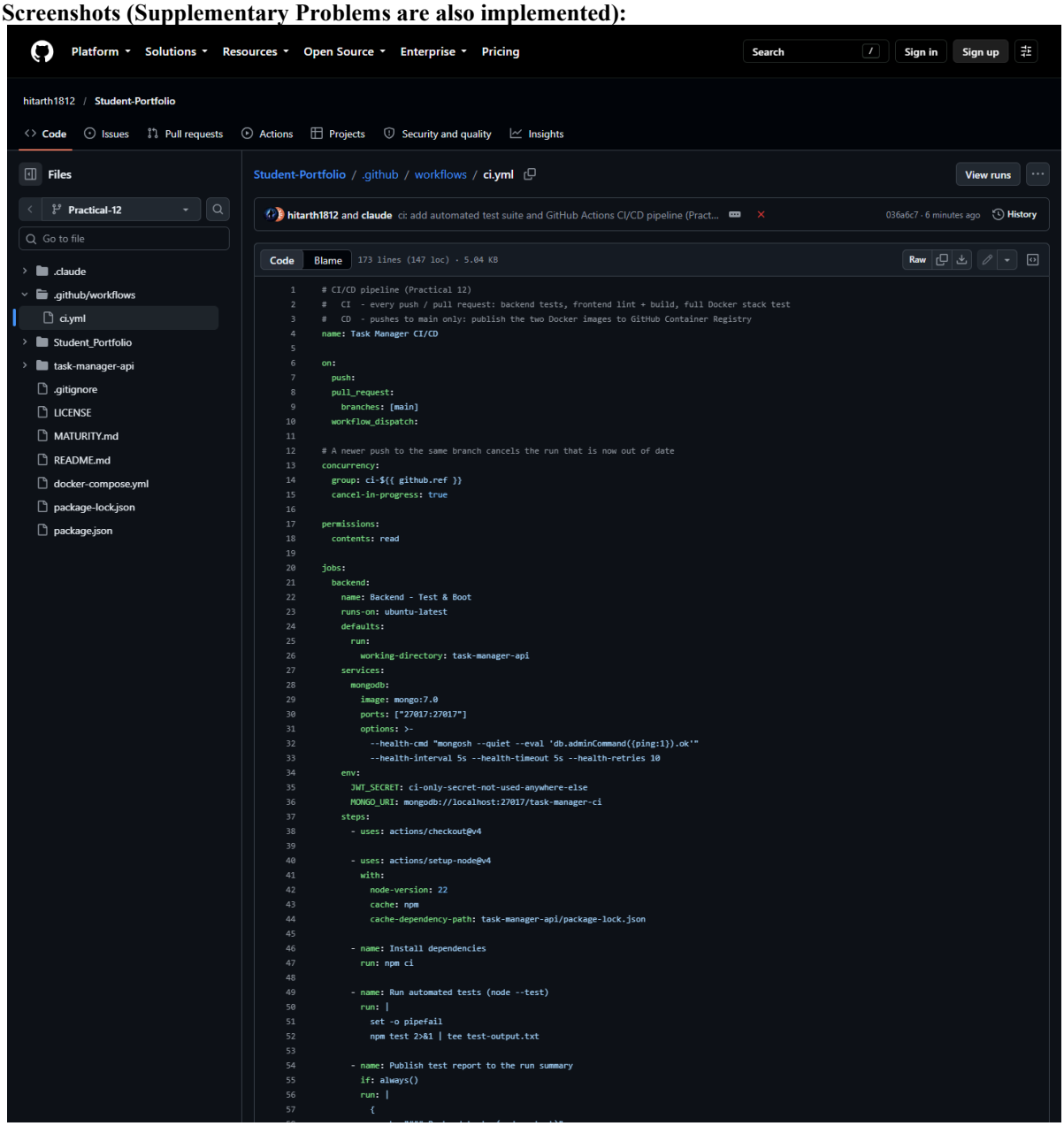


Figure 1: *The GitHub Actions workflow .github/workflows/ci.yml on the Practical-12 branch: triggers (push, pull request, manual), concurrency control, and the Backend job with a MongoDB service container.*

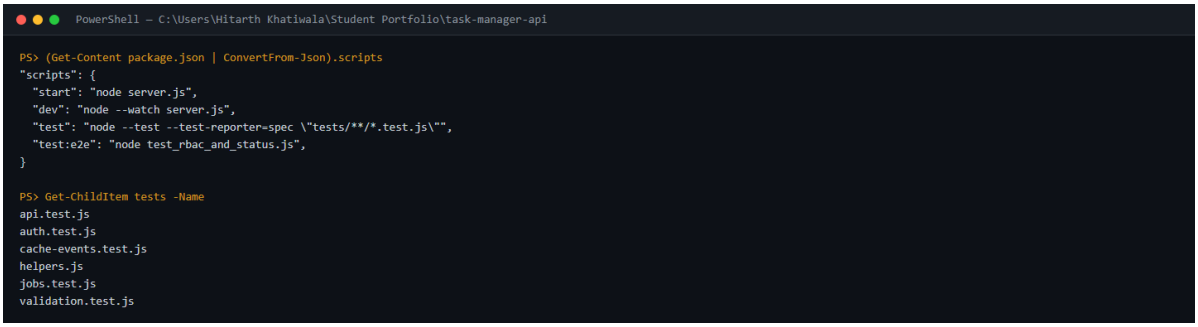


Figure 2: *task-manager-api package.json: npm test runs Node's built-in test runner (node --test) over tests/**/*.test.js; the six files hold 25 tests and need no extra test framework.*

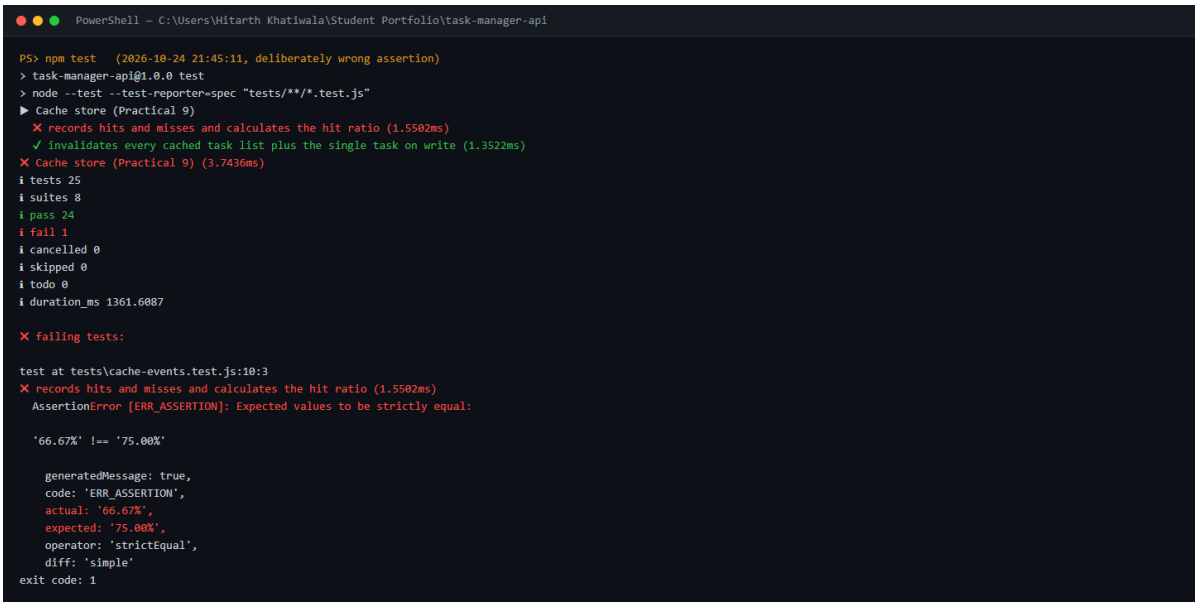


Figure 3: *Local npm test with a deliberately incorrect assertion: the hit-ratio test expects '75.00%' but 2 hits of 3 requests is '66.67%', so 24 tests pass, 1 fails and the process exits with code 1.*

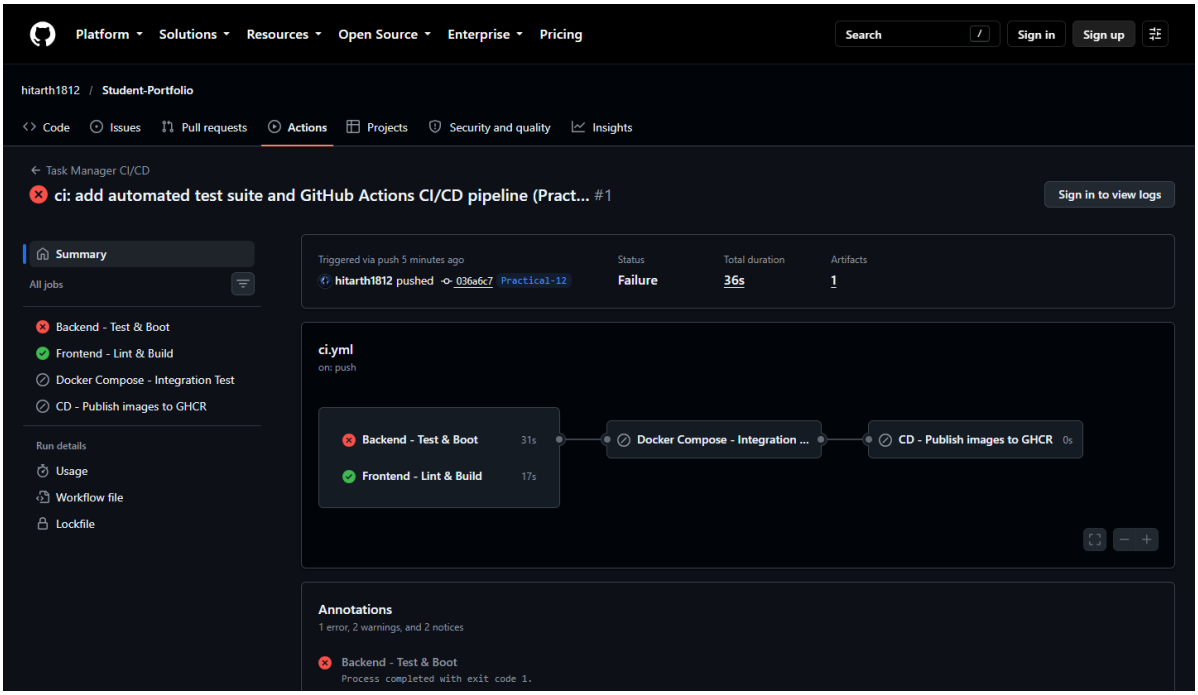


Figure 4: *CI run #1 on GitHub Actions for the same commit: Backend - Test & Boot fails ("Process completed with exit code 1"), so the dependent Docker integration and CD jobs are skipped; the independent Frontend job still passes.*

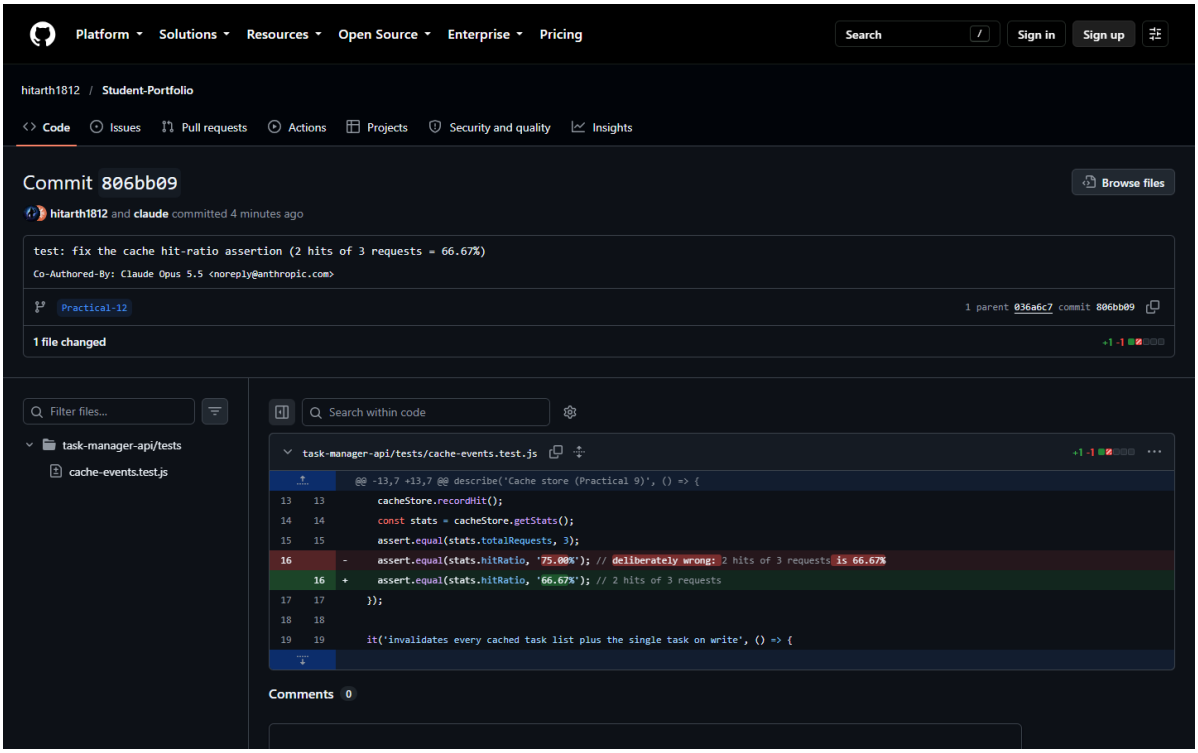


Figure 5: *The fix commit 806bb09 on GitHub: the expected value is corrected from '75.00%' to '66.67%'.*

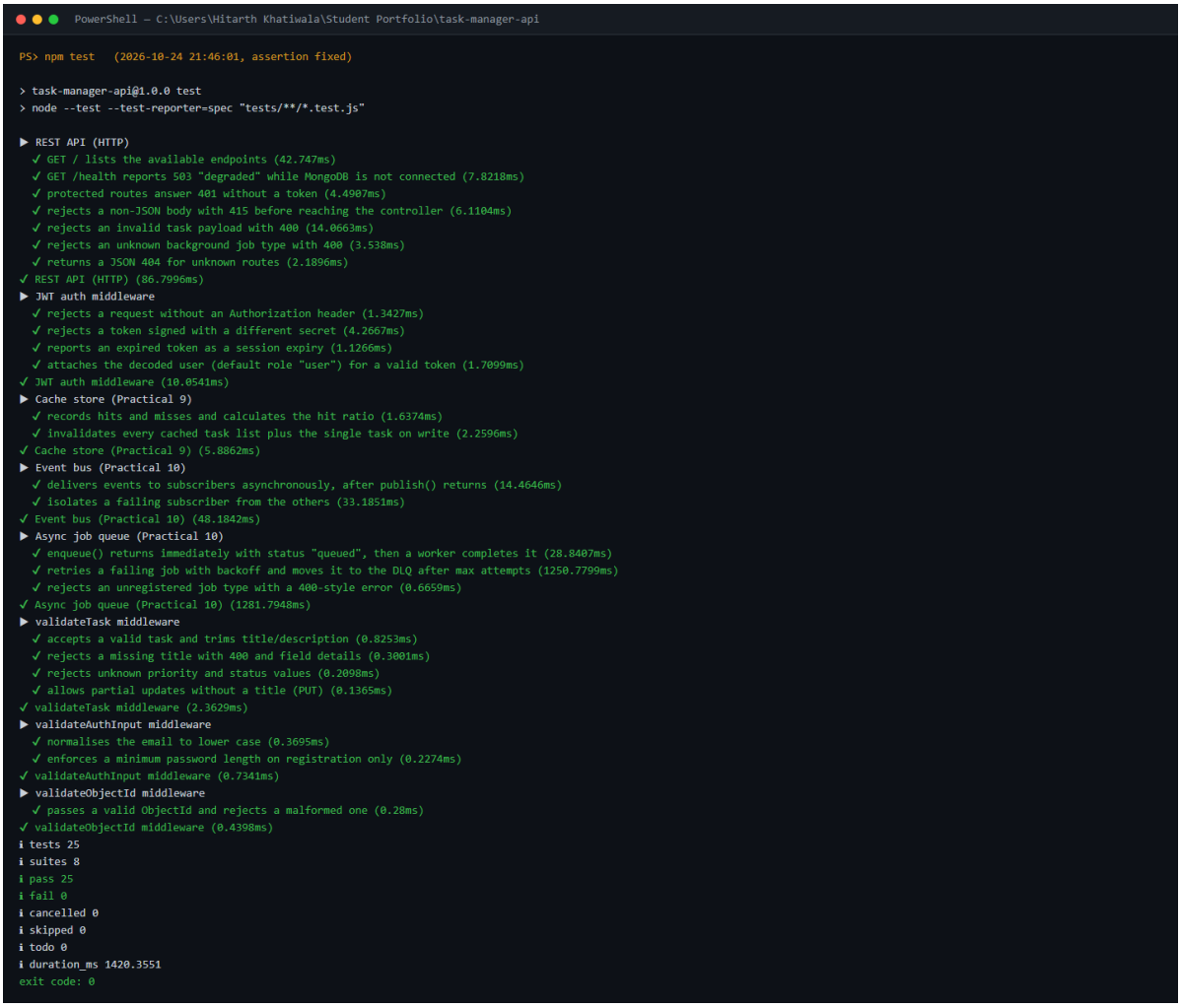


Figure 6: *Local npm test after the fix: all 25 tests in 8 suites pass (validation, JWT auth, cache, event bus, job queue with retry/DLQ, HTTP API) with exit code 0.*

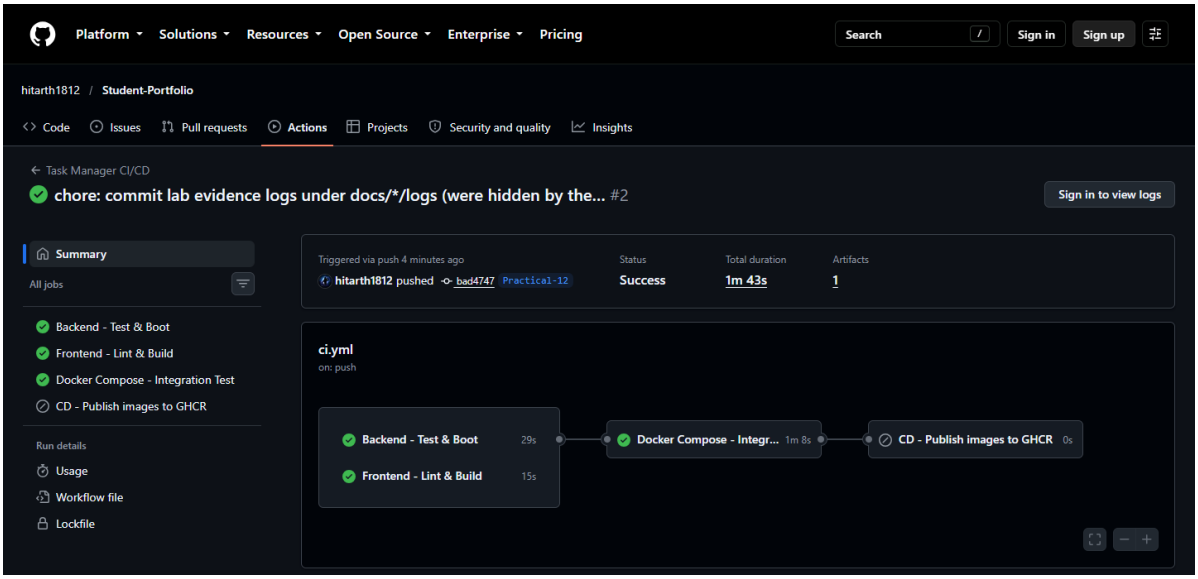


Figure 7: *CI run #2: Backend - Test & Boot (tests + booting the API against MongoDB), Frontend - Lint & Build and the Docker Compose integration test (full stack + 20-check smoke test) all succeed; CD - Publish to GHCR is skipped because it only runs on main.*

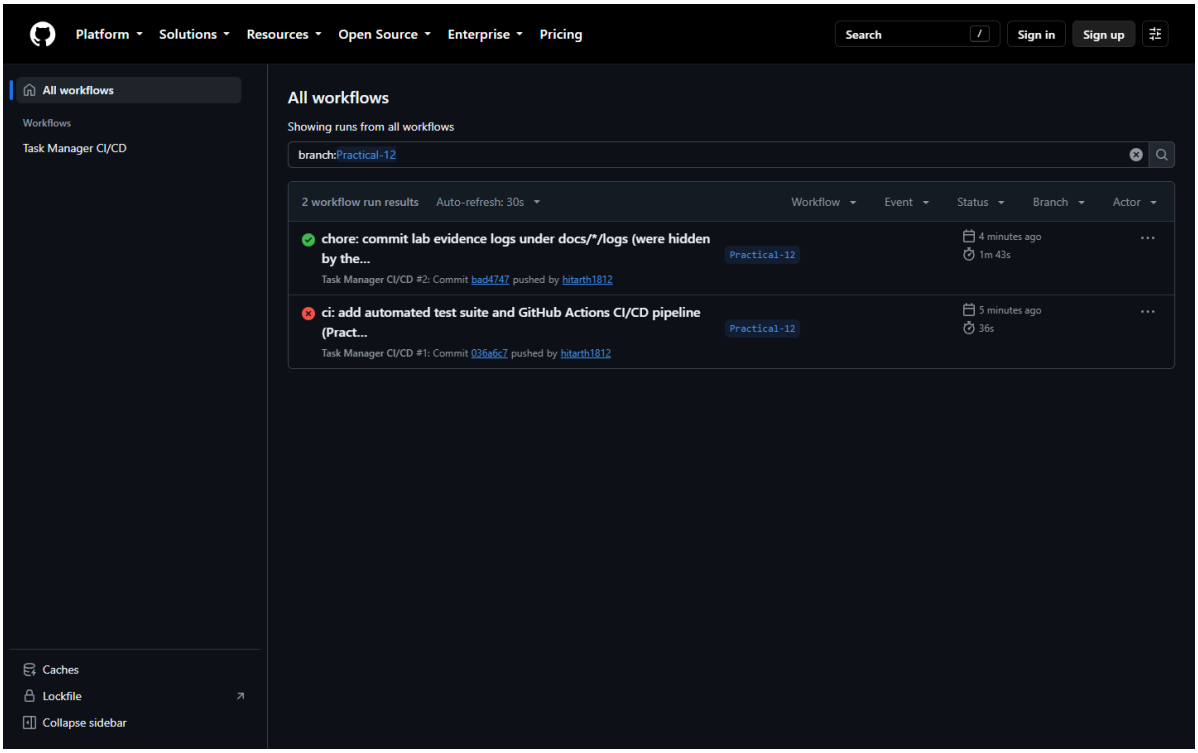


Figure 8: *CI-CD proof: the Actions history for the Practical-12 branch showing the failed run (#1, red) followed by the successful run (#2, green).*

Github Repo Link: https://github.com/hitarth1812/Student-Portfolio

Key Questions:

• What is the difference between Continuous Integration and Continuous Deployment, and which one does this practical actually demonstrate?
Ans: Continuous Integration is the habit of automatically building and testing the code every time a change is pushed, so that mistakes are spotted as soon as they are introduced. Continuous Deployment builds on this: any change that passes the whole pipeline is released to a live environment automatically, without a person pressing a button. What this practical shows is Continuous Integration - each push runs the backend tests, a lint check and production build of the frontend, and an integration test of the complete Docker Compose stack. The pipeline also has a stage that publishes versioned Docker images to GitHub Container Registry when code reaches main; that is Continuous Delivery of build artifacts, but nothing is deployed to a running server automatically, so the practical does not demonstrate full Continuous Deployment.

• Why does running tests automatically on every push catch problems earlier than testing manually before a release?
Ans: When tests run automatically on every push, the result arrives within a couple of minutes, while the change is small and the developer still remembers exactly what was edited, so the cause is easy to find and cheap to fix. The incorrect hit-ratio assertion in this practical, for example, was reported by the very first run after it was pushed. Manual testing just before a release happens much later and has to cover many changes at once, depends on someone remembering every check, and is never repeated in exactly the same way. A problem found at that stage is harder to trace back to its source and other work may already depend on it.

• What would happen to the pipeline if the test file path were incorrect - would the workflow fail at the same step as a genuine test failure?
Ans: Yes - the job would still turn red at the same step, "Run automated tests", because the command would exit with a non-zero code, but the reason would be completely different. With node --test, a path or pattern that points to a missing file makes the runner report an error about the file itself instead of executing any assertions, so the log would show a file-not-found or no-match message rather than an AssertionError such as expected '75.00%' and actual '66.67%'. That is why the log has to be read and not just the red or green status: a broken

configuration and a genuine bug both stop the pipeline, but only the second means the application code is wrong. It is also worth checking the summary line (tests 25, pass 25), because a pattern that silently matches no tests at all could look like a pass.